

Introducción a Rust

Técnicas de Programación Concurrente

Bienvenidos!

Sitio de la materia: <https://concurrentes-fiuba.github.io>

Discord:

Dinámica

Cronograma

Índice

- ¿Por qué Rust?
- Sintaxis
- Ownership
- Threads
- Testing

¿Por qué Rust?

- Objetivos de Rust: velocidad, seguridad y **conurrencia**.
- Se siente como un lenguaje expresivo de alto nivel, mientras que alcanza muy alta performance.
- Adhiere al principio de "zero-cost abstractions".
- Palabras de la comunidad: "empoderar a todos", extremadamente rápido, concurrencia sin miedo, retrocompatibilidad.

¿Por qué Rust?

Seguridad

- Prevenir el acceso a datos/memoria inválida, en tiempo de compilación (memory safe sin costo en tiempo de ejecución) (**buffer overflow**).
- No existen los "punteros sueltos" (**dangling pointers**), las referencias a datos inválidos son descartadas.
- Previene condiciones de carrera sobre los datos al usar concurrencia.
- Uno de los lenguajes recomendados como seguros por la casa blanca, NSA, etc.

Productividad

- ayudas del compilador, tipos de datos sofisticados, pattern matching, etc.
- Herramienta de construcción incorporada con gran biblioteca de paquetes (carga)

¿Por qué Rust?

oven-sh/bun

#30412 Rewrite Bun in Rust

2 comments 160 reviews 2k files +1009257 -4024

Jarred-Sumner • May 8, 2026 7k commits

READMEAGPL-3.0 license

pgrust

A Rust rewrite of Postgres that's faster than Postgres and ClickHouse.

Postgres18.3

regression suite46,066/46,066

versionv0.2

licenseAGPL-3.0

Try pgrust in your browser →

Discord

Updates

Issues

New:

How AI Changes the Economics of JIT Compilers

pgrust is a re-implementation of Postgres meant to show what Postgres would look like if it was built in 2026. It is wire compatible and even SQL dialect compatible with Postgres. It passes [all 46,066 tests in Postgres' regression suite](#). For the story of why we're building it, see [pgrust: rebuilding Postgres in Rust with AI](#).

utils speed-ups		
Utility	Speedup (vs. GNU)	Main optimization technique
sort	3.72x	Parallel merge sort
expand	1.80x	Optimized string buffer allocation
tr	1.58x	Complete logic refactoring
nl	1.57x	Streamlined line numbering

Hola Mundo!

```
fn main() {  
    println!("hola mundo!");  
}
```

Variables

```
// Las variables son inmutables por default
let t = true;

// Para hacerlas mutables usamos mut
let mut punto = (1_u8, 2_u8);
punto = (4, 3);

// Los tipos pueden ser inferidos como en los ejemplos anteriores,
// o explícitos. La asignación se denomina "binding"
let numero: i32 = 42;
```


Funciones

```
fn sumar_uno(a: i32) -> i32 {  
    a + 1  
}
```

// Closure

```
let plus_one = |a, b, c| { a + 1 }
```

// Closure como parámetro

```
fn map42 mapper: fn(i32) -> i32 -> i32 {  
    mapper(42)  
}
```

Structs

```
// Campos con nombre
#[derive(Debug)]
struct Persona {
    nombre: String,
    apellido: String
}

// 0 posicionales
struct NumeroImaginario(f64, f64);
```

Structs - Métodos

```
impl NumeroImaginario {  
    // "Método estático"  
    fn new(r:f64, i:f64) -> NumeroImaginario {  
        NumeroImaginario(r, i)  
    }  
  
    fn modulo(&self) -> f64 {  
        (self.0*self.0 + self.1*self.1).sqrt()  
    }  
}
```

Traits

```
trait MagnitudVectorial {  
    fn norma(&self) -> f64;  
}  
  
impl MagnitudVectorial for NumeroImaginario {  
    fn norma(&self) -> f64 {  
        self.modulo()  
    }  
}  
  
fn max(v1:&MagnitudVectorial, v2:&MagnitudVectorial) -> f64 {  
    v1.norma().max(v2.norma())  
}
```

Enums

```
enum Message {  
    Fire,  
    Move { x: i32, y: i32 },  
    Say(String),  
}
```

Pattern Matching

```
fn print_message(m:Message) {  
    match m {  
        Message::Fire => println!("Fire"),  
        Message::Move{ x:_, y} if y > 10 => println!("Corre hacia arriba"),  
        Message::Move{ x, y} => println!("Se mueve hacia {}, {}", x, y),  
        Message::Say(msg) => println!("Dice: {}", msg),  
    };  
}
```

Option y Result

```
enum Option<T> {  
    Some(T),  
    None  
}  
  
fn dividir(num: f64, den: f64) -> Option<f64> {  
    if den == 0.0 {  
        None  
    } else {  
        Some(num/den)  
    }  
}
```

Option y Result

```
enum Result<T, E> {  
    Ok(T),  
    Err(E)  
}  
  
fn contar_lineas(path:String) -> Result<u64, String> {  
    let file = File::open(path);  
    if !file.is_ok() {  
        return Err(String::from("error al abrir archivo"))  
    }  
    Ok(42)  
}
```


Option y Result

```
let d = dividir(3., 0.);  
d.unwrap() * 3 // pum!  
d.expect("dividió por 0") * 3 //pum!  
d.map(|n| n*3)
```

Option y Result

```
fn div_mul(n:f64, d:f64, m:f64) -> Option<f64>
{
    let d = dividir(n, d)?;
    Some(d * m)
}
```

Ownership

```
fn envejecer(mut p:Persona) {  
    p.edad += 20;  
    println!("{:?}", p)  
}  
  
fn main() {  
    let mut ariel = Persona { edad: 41 };  
    println!("{:?}", ariel);  
    envejecer(ariel);  
    println!("{:?}", ariel);  
}
```

Ownership

```
fn envejecer(p:&mut Persona) {  
    p.edad += 20;  
    println!("{:?}", p)  
}  
  
fn main() {  
    let mut ariel = Persona { edad: 41 };  
    println!("{:?}", ariel);  
    envejecer(&mut ariel);  
    println!("{:?}", ariel);  
}
```

Ownership

```
fn main() {  
    let p: &Persona;  
    {  
        let ariel = Persona { edad: 41 };  
        p = &ariel;  
    }  
    println!("{:?}", p);  
}
```

Ownership

Motivación: eliminar clases enteras de errores (punteros nulos, uso posterior a liberación, doble liberación, saturación de búfer (buffer overflow), invalidación de iterador, carreras de datos) al restringir programas válidos, sin incurrir en gastos generales de ejecución. Inspiración: C ++ RAI (resource acquisition is initialization)

- Un recurso se inicializa cuando se asigna.
- La inicialización la realizan los constructores de clases.
- La limpieza se realiza mediante destructores de clases.
- Los destructores se llaman automáticamente cuando el objeto sale de su alcance.

Ownership

- Cada valor en Rust tiene una variable, que es llamada su dueño (owner).
- Puede haber un único dueño a la vez. Cambiar de dueño es hacer un **move**
- El dueño puede prestar (borrow) múltiples referencias inmutables y **una única referencia mutable a la vez**.
- Cuando el dueño sale fuera de su scope, el valor es eliminado (dropped - trait Drop).

Borrow checker update

NLL vs Polonius Alpha

```
fn get_mut_or_default<'r, K: Hash + Eq + Copy, V: Default>(
    map: &'r mut HashMap<K, V>,
    key: K,
) -> &'r mut V {
    match map.get_mut(&key) {
        Some(value) => value,
        None => {
            map.insert(key, V::default());
            map.get_mut(&key).unwrap()
        }
    }
}
```


Heap

```
#[derive(Debug)]  
  
enum List<T> {  
    Cons(T, Box<List<T>>),  
    Nil,  
}  
  
fn main() {  
    let list: List<i32> =  
        List::Cons(1,  
            Box::new(List::Cons(2,  
                Box::new(List::Nil))));  
    println!("{:?}", list);  
}
```

Heap

- `Box<str> -> String`
- `Box<[T]> -> Vec` (además es iter)
- Box con múltiples referencias inmutables -> Rc
- Box con múltiples referencias inmutables desde múltiples threads -> Arc

Threads

```
println!("[PADRE] spawno hijo");  
let join_handle = thread::spawn(move || {  
    println!("[HIJO] spawnie");  
    thread::sleep(Duration::from_secs(2));  
    println!("[HIJO] me desperté")  
});  
println!("[PADRE] esperando hijo");  
join_handle.join();  
println!("[PADRE] terminó");
```

Testing

```
fn add(l: i32, r: i32) -> i32 {  
    l + r  
}  
#[cfg(test)]  
mod tests {  
    use super::*;  
    #[test]  
    fn it_adds_i32s() {  
        assert_eq!(5, add(3, 2));  
    }  
}
```

Referencias

- The Rust Programming Language, <https://doc.rust-lang.org/book/>
- Programming Rust: Fast, Safe Systems Development, 1st Edition, Jim Blandy, Jason Orendorff. 2017.
- Rust in Action, Tim McNamara, Manning. 2020.